

Caching

1 De cache structuur

1. Een 64KiB, direct mapped cache heeft 16 byte blocks. Je mag veronderstellen dat het geheugen byte geadresseerd is (dus 1 woord = 1 byte). Als adressen 32 bits lang zijn, hoeveel bits zijn er dan voorzien voor de tag, de index, en de offset in deze cache?

Solution:

$s - r$	r	w
tag	index	offset
16	12	4

- Cachegrootte 64KiB = 65536B = 2^{16} B, dus index en offset samen vormen 16 bits
- 32-bit adressen, en tag is wat overblijft, dus $32 - 16 = 16$, dus 16 bits.
- Blocksize 16B = 2^4 B, dus offset = 4 bits
- Index = $16 - 4$ bits = 12 bits,
of nog: $C = 2^{12}$ cache lijnen en dus 12 bits nodig om binnen alle lijnen te adresseren

2. Pas je antwoord aan indien de cache 4-way set associatief is in plaats van direct.

Solution:

$s - d$	d	w
tag	index	offset
18	10	4

- Het aantal “sets” wordt $4 = 2^2$ keer kleiner, dus index heeft 2 bits minder.

Of nog: $k = C / \#sets \Rightarrow \#sets = 2^d = C / k \Leftrightarrow k = 2^r / 2^d = 2^{r-d}$

3. Hoeveel bits hebben we voor de index nodig indien de cache volledig associatief is? Verklaar je antwoord.

Solution:

s	w
tag	offset
28	4

- Er blijft slechts 1 “set” over, dus 0 bits nodig als index binnen de cachelijnen.

2 Hit/miss in een 2-way LRU cache

Een 8 byte, 2-way set associatieve cache (met LRU replacement) met 2 byte blocks ontvangt requests voor de volgende (binaire) adressen:

0110, 0000, 0010, 0001, 0011, 0100, 1001, 0000, 1010, 1111, 0111.

Geef na elke access de inhoud van de cache door de bijgevoegde tabel in te vullen, voor elke set moet je de aanwezige tags weergeven.

Adres	Set 0	Set 1	Hit/Miss
0110	leeg/leeg	01/leeg	Hit /Miss
0000	/	/	Hit/Miss
0010	/	/	Hit/Miss
0001	/	/	Hit/Miss
0011	/	/	Hit/Miss
0100	/	/	Hit/Miss
1001	/	/	Hit/Miss
0000	/	/	Hit/Miss
1010	/	/	Hit/Miss
1111	/	/	Hit/Miss
0111	/	/	Hit/Miss

Solution:

$s - d$	d	w
tag	index	offset
2	1	1

Adres	Set 0	Set 1	Hit/Miss
0110	- - / - -	01 / - -	Hit /Miss
0000	00 / - -	01 / - -	Hit /Miss
0010	00 / - -	00 / 01	Hit /Miss
0001	00 / - -	00 / 01	Hit/ Miss
0011	00 / - -	00 / 01	Hit/ Miss
0100	01 / 00	00 / 01	Hit /Miss
1001	10 / 01	00 / 01	Hit /Miss
0000	00 / 10	00 / 01	Hit /Miss
1010	00 / 10	10 / 00	Hit /Miss
1111	00 / 10	11 / 10	Hit /Miss
0111	00 / 10	01 / 11	Hit /Miss

3 Cache hit rate

3M beslist Post-Its te maken door gele vierkanten op wit papier te printen, waarbij voor elk punt in het vierkant de CMYK (cyan, magenta, yellow, black) waarde moet worden ingevuld. 3M neemt je in dienst om drie implementaties te bekijken op een machine met een 2048-byte direct mapped data cache met 32 byte blokken/lijnen. Je krijgt volgende definities mee:

```
struct point_color {
    int c;
    int m;
    int y;
    int k;
};
struct point_color square[16][16];
register int i, j;
```

Veronderstel verder dat

- $\text{sizeof}(\text{int}) = 4$,
- square begint op memory address 0,
- de cache is initieel leeg,
- de enige memory accesses zijn naar de entries van de array square. De variabelen i en j zijn opgeslagen in de registers.
- het geheugen 4-byte geadresseerd wordt, dus 1 int = 1 woord.

Wat is het percentage van de writes in de volgende code die een hit veroorzaken?

Methode 1:

```
for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[i][j].c = 0;
        square[i][j].m = 0;
        square[i][j].y = 1;
        square[i][j].k = 0;
    }
}
```

Methode 2:

```
for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].y = 1;
        square[j][i].k = 0;
    }
}
```

Methode 3:

```
for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].y = 1;
    }
}

for (i=0; i<16; i++){
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].k = 0;
    }
}
```

Solution: 32B per block, dus 8 ints per block.

Er zijn per 1e index binnen de array 64 adressen (4 ints per struct en 16 structs (2e index) per 1e index). Het adres (in woorden uitgedrukt) van de kleurcomponenten van square is dus

$$\text{addr}(\text{square}[i][j]) = 64i + 4j + \begin{cases} 0 & c \\ 1 & m \\ 2 & y \\ 3 & k \end{cases}$$

Bepaal hierna in welke volgorde de absolute adressen (in woorden uitgedrukt) worden bezocht:

1.	Index	[0][0]	[0][0]	[0][0]	[0][0]	[0][1]	[0][1]	[0][1]	[0][1]	[0][2]	[0][2]	Hit rate: 7/8 = 87.5%
	Kleur:	c	m	y	k	c	m	y	k	c	m	
	Adres:	0	1	2	3	4	5	6	7	8	9	
	Hits:	M	H	H	H	H	H	H	H	M	H	

2.	Index	[0][0]	[0][0]	[0][0]	[0][0]	[1][0]	[1][0]	[1][0]	[1][0]	[2][0]	[2][0]	Hit rate: 3/4 = 75%
	Kleur:	c	m	y	k	c	m	y	k	c	m	
	Adres:	0	1	2	3	64	65	66	67	128	129	
	Hits:	M	H	H	H	M	H	H	H	M	H	

Zit 0...7 nog in de cache wanneer [0][1]=4...7 wordt bezocht en volgen er dus later nog extra hits? Nee, want [8][0]=512...515 wordt eerst bezocht, en dit mapt op dezelfde set want cache size = 2048B = 512 woorden.

3.	Index	[0][0]	[1][0]	[2][0]	[3][0]	Hit rate: 0%
	Kleur:	y	y	y	y	
	Adres:	2	66	130	194	
	Hits:	M	M	M	M	

	Index	[0][0]	[0][0]	[0][0]	[1][0]	[1][0]	[1][0]	[2][0]	[2][0]	[2][0]	[3][0]	Hit rate: 2/3 = 66.6%
	Kleur:	c	m	k	c	m	k	c	m	k	c	
	Adres:	0	1	3	64	65	67	128	129	131	192	
	Hits:	M	H	H	M	H	H	M	H	H	M	

Samen geeft dit $1/4 \cdot 0 + 3/4 \cdot 2/3 = 2/4 = 50\%$

Zelfde opmerking als bij methode 2.

4 Cache replacement algoritmen

Ga voor het tree-based en MRU-based pseudo LRU algoritme volgende twee zaken na en bewijs, indien nodig, deze eigenschappen:

1. Stel dat w_i het i -de meest recent gebruikte woord is, voor $i = 1$ tot de set size k . Bij LRU kan enkel woord w_k vervangen worden. Welke woorden kunnen er allemaal vervangen worden indien we een pseudo LRU hanteren?

Solution:

- MRU
 - w_1 kan niet vervangen worden, omdat de corresponderende bit op 0 wordt gezet.
 - Vanaf w_2 kunnen de woorden vervangen worden. Voorbeeld van een set waarbij A als 2-de recent gebruikte woord vervangen:

A	B	C	D
0	1	1	0

Access A

A	B	C	D
1	1	1	0

Access D

A	B	C	D
0	0	0	1

Access E

E	B	C	D
1	0	0	1

- Tree based: w_1 tot en met $w_{\log_2 k}$ kan niet worden vervangen. Dit kan je makkelijk inzien, door een element X op te vragen. Alle $\log_2 k$ bits op het pad naar X worden op 1 gezet. X wordt pas vervangen als die allemaal terug 0 zijn. Per access kan ten hoogste 1 bit op 0 gezet worden; bij het passeren van een 1 wordt het pad naar X niet verder afgelopen.

2. Stel dat een bepaald blok B (d.i., een bepaalde lijn) niet meer gebruikt wordt vanaf een zeker ogenblik. Hoe lang kan blok B dan nog aanwezig blijven in de cache? Anders gezegd: hoeveel replacements kan B maximaal ‘overleven’?

Solution:

- MRU: maximaal $2k - 3$ replacements overleven. Stel dat X wordt opgevraagd; nu kunnen er maximaal $k - 1$ replacements gebeuren, waarna de bit van X op 0 wordt gezet. Hierbij worden alle bits geflipt, en staan er dus $k - 1$ op 0. Na ten hoogste $k - 2$ replacements is het de beurt aan X om vervangen te worden.
- Tree based:
 - $k = 2$: dit is LRU, dus maximaal 1 replacements kunnen overleefd worden.
 - $k > 2$: vraag tussen elke replacement de dichtste buur van X op, dan wordt X nooit vervangen. Dus ∞ .

5 Access times en hit rates

Laat T_1 de access tijd voorstellen indien een woord zich in de $L1$ cache bevindt. Indien het woord enkel in de $L2$ cache aanwezig is en niet in de $L1$ cache, dan is er een tijd T_2 nodig om het woord over te brengen naar de $L1$ cache. Tenslotte, is er een tijd T_M nodig om een woord van het main geheugen naar de $L2$ cache te verplaatsen. Stel dat $T_1 = 20$ nsec, $T_2 = 100$ nsec en $T_M = 400$ nsec.

1. Indien we respectievelijk een hit rate van .9 en .5 hebben in de $L1$ en $L2$ cache, wat is dan de gemiddelde access time?

Solution:	90%		20ns	$0.9 \times 20\text{ns}$	=	18ns
	10%	50%	$100 + 20\text{ns}$	$0.05 \times 120\text{ns}$	=	6ns
		50%	$400 + 100 + 20\text{ns}$	$0.05 \times 520\text{ns}$	=	26ns
						50ns

2. Indien de hit rate in de $L2$ cache .6 is, wat is dan de hit rate in de $L1$ cache indien de gemiddelde access time gelijk is aan 33 nsec?

Solution:	x		20ns	$x \times 20\text{ns}$	=	$x20\text{ns}$
	$(1 - x)$	60%	$100 + 20\text{ns}$	$x \times 0.6 \times 120\text{ns}$	=	$(1 - x)72\text{ns}$
		40%	$400 + 100 + 20\text{ns}$	$x \times 0.4 \times 520\text{ns}$	=	$(1 - x)208\text{ns}$
						33ns

$$x(20 - 72 - 208) + 72 + 208 = 33 \Rightarrow x = 247/260 = 0.95$$

6 Direct mapping versus set-associative mapping met random replacement

Beschouw het volgende stuk code uit de theorieslides.

```
float a[1024], b[1024];
for (i=0; i<1024; i++)
    sum += a[i] * b[i];
```

Veronderstel eveneens:

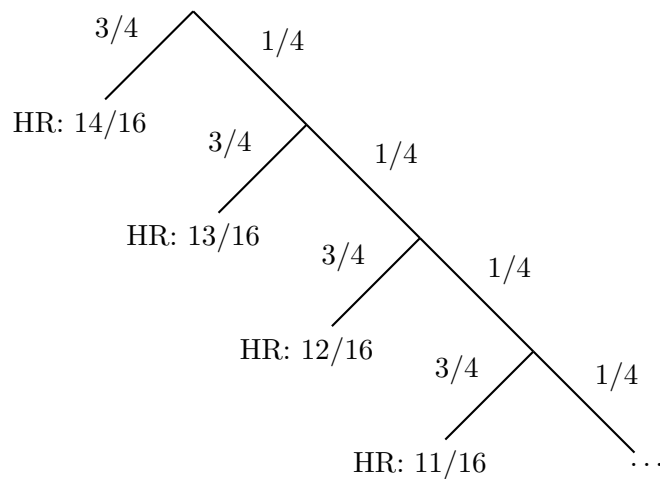
- Cache size: 4 KByte, Block size: 32 byte, Word size: 4 byte
- Float: 4 byte
- b zit vlak na a in het geheugen
- i en sum zijn registers

1. Wat is de hit rate als we direct-mapping veronderstellen ?

Solution: 0, zie de theorie slides.

2. Als we een 4-way set-associatief geheugen met random replacement veronderstellen, bereken de (benaderde) hit rate.

Solution: Van zodra zowel a als b in de cache zitten, zijn er hits. Tot dat, kan telkens b in de cache wordt gezet, a vervangen worden en omgekeerd. Bekijk de eerste 8 iteraties, dit komt overeen met een block van a en een block van b. In de volgende boom komt een linkertak overeen met "a en b zitten in de cache" en een rechtertak met "a vervangt b of b vervangt a".



$$\sum_{i=0}^{\infty} \frac{3}{4} \left(\frac{1}{4}\right)^i \frac{14-i}{16}$$

De eerste 4 termen hiervan uitrekenen zijn al voldoende om de 85% van de theorieslides te benaderen.

3. Pas je antwoord op vorige 2 vragen aan, als je weet dat beide arrays elk slechts 256 elementen bevatten. Doe dit eveneens voor 1536.

Solution: Bij 256 elementen liggen de arrays 1/4e van de cache size uit elkaar (en worden overeenkomstige elementen dus op dezelfde set gemapt).

Bij 1536 liggen ze 6/4e van de cache size uit elkaar: dat is een veelvoud van 1/ke = 1/4e van de cache size uit elkaar en worden overeenkomstige elementen dus weer op dezelfde sets gemapt.

- Bij direct mapping is er nu geen thrashing meer, dus een hit rate van 7/8.
- Bij 4-way set associative blijft het antwoord hetzelfde, nl. ongeveer 85%.

7 Benchmarking

Teken een grafiek die een cache benchmark, zoals beschreven op p31 van de theoriecursus, illustreert. We beschouwen een systeem dat enkel een L1 cache heeft; eventuele TLB en paging effecten hoeven niet weergegeven te worden. De grootte van de L1 cache is 32 KByte, een cache lijn bevat 64 bytes en er wordt 8-way associativiteit verondersteld. Een hit heeft een access tijd van 50 ns, een miss kost 500 ns extra. Maak de grafiek, met op de x-as de stride-grootte (logaritmische schaal) en de access tijd op de y-as, voor arraygroottes van 4 tot 512 Kbyte.

Solution:

